

# **Loading and Handling Data in R**

## **Data Analytics using R – Chapter 3**

Anil

2026-07-14

# Challenges of Analytical Data Processing

## Data Formats

- Data comes from varied sources (CSV, Excel, SQL, web, etc.)
- R supports many formats via packages (ODBC, readr, etc.)

## Data Quality

- Missing values (NA), outliers, inconsistent data
- R provides functions to clean and impute

## Project Scope

- Time, cost, and data volume must be planned

## Stakeholder Expectation Management

- Present clear results; hide technical details (p-values, etc.)

# Expressions in R

## Arithmetic Operators

| Operation        | Operator | Example  | Result   |
|------------------|----------|----------|----------|
| Addition         | +        | 4 + 8    | 12       |
| Subtraction      | -        | 10 - 3   | 7        |
| Multiplication   | *        | 7 * 8    | 56       |
| Division         | /        | 8 / 3    | 2.666667 |
| Exponentiation   | ^ or **  | 2 ^ 5    | 32       |
| Modulus          | %%       | 5 %% 3   | 2        |
| Integer division | %/%      | 5 %/% 2  | 2        |
| Square root      | sqrt()   | sqrt(25) | 5        |

# Logical Values

```
8 < 4  
3 * 2 == 5  
3 * 2 == 6  
F == FALSE  
T == TRUE
```

```
[1] FALSE  
[1] FALSE  
[1] TRUE  
[1] TRUE  
[1] TRUE
```

## Using Logical Operators with Vectors

```
x <- c(1:10)  
x[(x > 7) | (x < 5)] # OR
```

```
[1] 1 2 3 4 8 9 10
```

```
x[(x > 7) & (x < 10)] # AND
```

```
[1] 8 9
```

# Dates in R

## System Date & Time

```
Sys.Date()  
Sys.time()  
Sys.timezone()
```

```
[1] "2017-01-13"  
[1] "2017-01-13 10:54:37 IST"  
[1] "Asia/Calcutta"
```

## Formatting and Conversion

```
today <- Sys.Date()  
format(today, format = "%B %d %Y")
```

```
[1] "January 13 2017"
```

```
CustomDate <- "2016-01-13"  
class(CustomDate)  
CustDate <- as.Date(CustomDate)  
class(CustDate)
```

```
[1] "character"  
[1] "Date"
```

## Difference Between Dates

```
strDates <- c("08/15/1947", "01/26/1950")  
dates <- as.Date(strDates, "%m/%d/%Y")  
dates[2] - dates[1]
```

```
Time difference of 895 days
```

# Variables

```
Var <- 50  
Var  
Var + 10  
Var / 2  
Var <- "R is a Statistical Programming Language"  
Var  
Var <- TRUE  
Var
```

```
[1] 50  
[1] 60  
[1] 25  
[1] "R is a Statistical Programming Language"  
[1] TRUE
```

- Variables can be reassigned to different data types.

# Functions: `sum()`, `min()`, `max()`

## `sum()`

```
sum(1, 2, 3)
sum(1, 5, NA, na.rm = FALSE)
sum(1, 5, NA, na.rm = TRUE)
```

```
[1] 6
[1] NA
[1] 6
```

## `min()` and `max()`

```
min(1, 2, 3, NA, na.rm = TRUE)
max(44, 78, 66)
```

```
[1] 1
[1] 78
```

# String Functions

## rep() – Repeat

```
rep("statistics", 3)
```

```
[1] "statistics" "statistics" "statistics"
```

## grep() – Pattern Search

```
grep("statistical", c("R", "is", "a", "statistical", "language"), fixed = TRUE)
```

```
[1] 4
```

## toupper() / tolower()

```
toupper("statistics")  
tolower("STATISTICS")
```

```
[1] "STATISTICS"  
[1] "statistics"
```

## substr() – Extract Substring

```
substr("statistics", 7, 9)
```

```
[1] "tic"
```

# Missing Values Treatment

| Function                   | Description                     |
|----------------------------|---------------------------------|
| <code>is.na(x)</code>      | Returns TRUE for missing values |
| <code>na.omit(x)</code>    | Removes missing values          |
| <code>na.exclude(x)</code> | Removes, but keeps attributes   |
| <code>na.fail(x)</code>    | Fails if any NA present         |
| <code>na.pass(x)</code>    | Returns unchanged object        |

```
is.na(c(4, 5, NA))  
na.omit(c(4, 5, NA))
```

```
[1] FALSE FALSE TRUE  
[1] 4 5  
attr(,"na.action")  
[1] 3  
attr(,"class")  
[1] "omit"
```

# Vectors

## Creating Vectors

```
c(4, 7, 8)           # numeric
c("R", "is", "powerful") # character
c(TRUE, FALSE, TRUE)  # logical
```

## Sequence Vectors

```
1:5
seq(1, 10, 2)
seq(10, 1, by = -2)
```

```
[1] 1 2 3 4 5
[1] 1 3 5 7 9
[1] 10 8 6 4 2
```

## Accessing Elements

```
x <- c("R", "is", "a", "programming", "language")
x[1]
x[c(1, 5)]
x[1:4]
```

```
[1] "R"
[1] "R" "language"
[1] "R" "is" "a" "programming"
```

# Vector Names and Math

## Assigning Names

```
v <- 1:5  
names(v) <- c("r", "is", "a", "programming", "language")  
v["programming"]
```

```
programming  
4
```

## Vector Arithmetic

```
x <- c(4, 7, 8)  
x + 1  
x * 2  
sin(x)
```

```
[1] 5 8 9  
[1] 8 14 16  
[1] -0.7568025 0.6569866 0.9893582
```

## Vector Recycling

```
c(1, 2, 3) + c(4, 5, 6, 7, 8, 9)
```

```
[1] 5 7 9 8 10 12
```

# Matrices

## Creating Matrices

```
matrix(1, 3, 4)           # 3x4 matrix filled with 1
a <- seq(10, 90, by = 10)
matrix(a, 3, 3)           # fill column-wise
```

|      | [,1] | [,2] | [,3] |
|------|------|------|------|
| [1,] | 10   | 40   | 70   |
| [2,] | 20   | 50   | 80   |
| [3,] | 30   | 60   | 90   |

## Using dim()

```
a <- 1:9
dim(a) <- c(3, 3)
a
```

|      | [,1] | [,2] | [,3] |
|------|------|------|------|
| [1,] | 1    | 4    | 7    |
| [2,] | 2    | 5    | 8    |
| [3,] | 3    | 6    | 9    |

# Matrix Access

```
mat <- matrix(1:12, 3, 4)
mat
mat[2, 3]      # row 2, col 3
mat[3, ]       # row 3
mat[, 2]        # column 2
mat[, 2:3]      # columns 2 and 3
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
[1] 8
[1] 3 6 9 12
[1] 4 5 6
      [,1] [,2]
[1,]    4    7
[2,]    5    8
[3,]    6    9
```

# Visualising Matrices: Contour, Perspective, Heatmap

## Contour Plot

```
contour(volcano)
```

*(see Figure 3.7)*

## Perspective Plot

```
persp(volcano)
```

*(see Figure 3.8)*

## Heatmap

```
image(volcano)
```

*(see Figure 3.9)*

# Factors

## Creating Factors

```
HouseColor <- c('red', 'green', 'blue', 'yellow', 'red', 'green', 'blue', 'blue')
types <- factor(HouseColor)
types
levels(types)
as.integer(types)
```

```
[1] red   green blue  yellow red   green blue  blue
Levels: blue green red  yellow
[1] "blue" "green" "red"   "yellow"
[1] 3 2 1 4 3 2 1 1
```

## Plotting with Factors

```
NoofStudents <- c(12, 14, 12, 13)
AverageScore <- c(70, 80, 90, 95)
plot(NoofStudents, AverageScore, pch = as.integer(types))
legend("topright", levels(types), pch = 1:4)
```

(see *Figures 3.10–3.12*)

# Lists

## Creating Lists

```
emp <- list(EmpName = "Alex", EmpUnit = "IT", EmpSal = 55000)
emp
```

```
$EmpName
[1] "Alex"

$EmpUnit
[1] "IT"

$EmpSal
[1] 55000
```

## Without Tags

```
Emplist <- list("Alex", "IT", 55000)
Emplist
```

```
[[1]]
[1] "Alex"

[[2]]
[1] "IT"

[[3]]
[1] 55000
```

# List Tags and Values

## Retrieving Tags and Values

```
names(emp)
unlist(emp)
emp[["EmpName"]]
```

```
[1] "EmpName" "EmpUnit" "EmpSal"
EmpName    EmpUnit    EmpSal
"Alex"      "IT"        "55000"
[1] "Alex"
```

## Add / Delete Elements

```
emp$EmpDesg <- "Software Engineer"
emp$EmpUnit <- NULL # delete
```

## Size of List

```
length(emp)
```

```
[1] 3
```

## Recursive List (list within a list)

```
EmpList <- list(emp, emp1)
```

# Exploring a Dataset

| Function               | Description                             |
|------------------------|-----------------------------------------|
| <code>names()</code>   | Variable names                          |
| <code>summary()</code> | Summary statistics                      |
| <code>str()</code>     | Internal structure                      |
| <code>head()</code>    | First n rows (default 6)                |
| <code>tail()</code>    | Last n rows (default 6)                 |
| <code>class()</code>   | Class of object                         |
| <code>dim()</code>     | Dimensions (rows, columns)              |
| <code>table()</code>   | Frequency table of categorical variable |

## Example on Orange dataset

```
data(Orange)
names(Orange)
summary(Orange)
str(Orange)
head(Orange, 3)
tail(Orange, 3)
dim(Orange)
table(Orange$Tree)
```

# Conditional Manipulation

```
# Display first row  
TD[1, ]  
  
# Display first column  
TD[, 1]
```

(See Figure 3.16)

# Merging Data

## merge( ) Function

```
S <- data.frame(RN = c('A','B','C'), RM = c(10,20,30))  
T <- data.frame(TN = c('A','B','C'), TM = c(10,20,30))  
E <- merge(S, T) # natural join
```

## Join Types

- `all = TRUE` – full outer join
- `all.x = TRUE` – left outer join
- `all.y = TRUE` – right outer join

*(See Figures 3.17–3.18)*

# Aggregating and Grouping

## aggregate()

```
# Assuming 'Fruit_data.csv' with columns Fruit.Name, Fruit.Price  
S <- read.csv("Fruit_data.csv")  
aggregate(S$Fruit.Price ~ S$Fruit.Name, S, mean)
```

*(See Figure 3.19)*

## tapply()

```
tapply(S$Fruit.Price, S$Fruit.Name, sum)
```

*(See Figure 3.20)*

# Simple Analysis Workflow

## Input

```
Fruit <- read.csv("Fruit_data.csv")
```

## Describe Data Structure

```
names(Fruit)
str(Fruit)
summary(Fruit)
head(Fruit)
tail(Fruit)
```

## Describe Variables

```
table(Fruit$Fruit.Name)
hist(Fruit$Fruit.Price)
boxplot(Fruit$Fruit.Price ~ Fruit$Fruit.Name)
plot(Fruit$Fruit.Price, Fruit$Fruit.Quantity)
```

# Reading CSV and Spreadsheets

## CSV

```
read.csv("Hardware.csv")  
read.table("Hardware.csv", header = TRUE, sep = ",")
```

## Excel (using xlsx package)

```
install.packages("xlsx")  
library(xlsx)  
read.xlsx("Softdrink.xlsx", 1) # sheet index
```

## Practical Example: SampleSuperstore

```
InputData <- read.csv("d:/SampleSuperstore.csv")  
GroupedInputData <- aggregate(InputData$Sales ~ InputData$Category, InputData, sum)  
GroupedInputData
```

|   | InputData\$Category | InputData\$Sales |
|---|---------------------|------------------|
| 1 | Furniture           | 156514.4         |
| 2 | Office Supplies     | 132600.8         |
| 3 | Technology          | 168638.0         |

# Reading from Packages

## `library()` and `data()`

```
library(datasets)
data()           # list all datasets
data(Orange)    # load Orange dataset
Orange
```

# Reading Web Data

## Packages for Web Data

- RCurl – download files, web scraping
- XML – parse XML/HTML
- WDI – World Bank data
- quantmod – Yahoo Finance

## Web Scraping Example

```
library(RCurl)
library(XML)
url <- "http://example.com"
html <- getURL(url)
parsed <- htmlTreeParse(html, useInternalNodes = TRUE)
```

# Reading JSON

## Using rjson package

```
install.packages("rjson")
library(rjson)
output <- fromJSON(file = "d:/Jsondoc.json")
output
JSONDataFrame <- as.data.frame(output)
JSONDataFrame
```

## Sample JSON data:

```
{
  "EMPID": ["1001", "2001", ...],
  "Name": ["Ricky", "Danny", ...],
  "Dept": ["IT", "Operations", ...]
}
```

## Output:

|   | EMPID | Name  | Dept       |
|---|-------|-------|------------|
| 1 | 1001  | Ricky | IT         |
| 2 | 2001  | Danny | Operations |
|   | ...   |       |            |

# Reading XML

## Using XML package

```
install.packages("XML")
library(XML)
output <- xmlParse(file = "d:/XMLFile.xml")
rootnode <- xmlRoot(output)
rootsize <- xmlSize(rootnode) # number of nodes
xmlToDataFrame("d:/XMLFile.xml")
```

## Sample XML:

```
<RECORDS>
  <EMPLOYEE>
    <EMPID>1001</EMPID>
    <EMPNAME>Merrilyn</EMPNAME>
    ...
  </EMPLOYEE>
</RECORDS>
```

## Output data frame:

|     | EMPID | EMPNAME  | SKILLS           | DEPT            |
|-----|-------|----------|------------------|-----------------|
| 1   | 1001  | Merrilyn | MongoDB          | ComputerScience |
| 2   | 1002  | Ramya    | PeopleManagement | HumanResources  |
| ... |       |          |                  |                 |

# R GUIs for Data Input

| GUI                           | Description                              |
|-------------------------------|------------------------------------------|
| <b>RCommander<br/>(Rcmdr)</b> | Simple interface, many plug-ins          |
| <b>Rattle</b>                 | Data mining GUI                          |
| <b>RKward</b>                 | Transparent front-end,<br>cross-platform |
| <b>JGR</b>                    | Java GUI for R                           |
| <b>Deducer</b>                | Menu system, works with JGR              |

# Using R with Databases

## RODBC – MS Access / SQL Server

```
library(RODBC)
conn <- odbcConnect(dsn = "servername", uid = "", pwd = "")
query <- "SELECT * FROM table"
data <- sqlQuery(conn, query)
odbcClose(conn)
```

## MySQL – RMySQL

```
library(RMySQL)
conn <- dbConnect(MySQL(), user = "", password = "", dbname = "", host = "")
res <- dbSendQuery(conn, "SELECT * FROM table")
data <- fetch(res, n = -1)
dbDisconnect(conn)
```

## PostgreSQL – RPostgreSQL

Similar functions: `dbConnect()`, `dbDisconnect()`.

## SQLite – RSQLite

```
library(RSQLite)
conn <- dbConnect(SQLite(), dbname = "myDB.sqlite")
```

# JasperDB and Pentaho

## JasperReports Server

- Java library: **RevoConnectR**
- Web services: **RevoDeployR**

## Pentaho Data Integration (PDI)

- **R Script Executor** – integrates R with PDI
- Allows using R scripts within ETL pipelines

# Log Analysis

## Reading Log Files

```
LOGS <- read.table("log.log", sep = " ", header = FALSE)  
head(LOGS)
```

## Analysis and Visualisation

```
summary(LOGS)  
barplot(table(LOGS[, column]))
```

## Save Plot

```
png("test.png", width = 1024, height = 1024)  
barplot(table(LOGS[, column]))  
dev.off()
```

# Multiple Choice Questions (Sample)

1. Which is **not** a challenge for analytical data processing?

1. *Data Formats* (b) *Project Scope* (c) *Data Quality* (d) **Data Input** → **Answer: (d)**

2. Which function loads a package?

1. *load()* (b) **library()** (c) *data()* (d) *install()* → **Answer: (b)**

3. Which function lists available datasets?

1. *library()* (b) *data(datasetname)* (c) **data()** (d) *install()* → **Answer: (c)**

4. Which package reads World Bank data?

1. *RCurl* (b) *XML* (c) **WDI** (d) *quantmod* → **Answer: (c)**

5. Which function returns number of categorical values?

1. **table(dataset\$variable)** (b) *table(dataset.variable)* (c) *table(dataset)* (d) *table(variables)* → **Answer: (a)**

# Summary

- R supports vectors, matrices, lists, and factors for data storage.
- Missing values (NA) can be handled with `is.na()`, `na.omit()`, etc.
- Text manipulation: `grep()`, `substr()`, `strsplit()`, `paste()`.
- Data exploration: `summary()`, `str()`, `head()`, `tail()`, `names()`, `dim()`.
- Data import: CSV (`read.csv`), Excel (`read.xlsx`), JSON (`rjson`), XML (`XML`).
- Web data: `RCurl`, `XML`, `WDI`, `quantmod`.
- Database connections: `RODBC`, `RMySQL`, `RPostgreSQL`, `RSQLite`.
- Aggregation: `aggregate()`, `tapply()`; merging: `merge()`.
- Visualisations: `hist()`, `boxplot()`, `plot()`, `contour()`, `persp()`, `image()`.

# Key Terms

- **CSV** – Comma Separated Values file.
- **Data frame** – Tabular data structure in R.
- **Factor** – Categorical variable with levels.
- **GUI** – Graphical User Interface.
- **JSON** – JavaScript Object Notation.
- **XML** – eXtensible Markup Language.
- **ODBC** – Open Database Connectivity.
- **Web scraping** – Extracting data from websites.

